



explain Invocable method in detail with multiple examples also explain the examples in salesforce apex

In **Salesforce Apex**, the `@InvocableMethod` annotation is used to allow **Apex methods to be called from declarative tools** like:

- **Flows (Lightning Flow / Flow Builder)**
- **Process Builder**

This makes Apex classes/methods available for use without writing code in the Flow/Process Builder UI — great for admins and low-code developers.

◆ Syntax of @InvocableMethod

```
apex

global class MyClass {
    @InvocableMethod
    public static void myMethod(List<CustomType> inputList) {
        // logic here
    }
}
```

◆ Rules and Key Points

- The method must be **public** or **global** and **static**.
- It must accept **a list of a class or a primitive type**.
- That class (if used) must be an **inner class** annotated with `@InvocableVariable`.
- The return type must be **void** or **List<AnotherCustomType>** (if output is needed).
- Used primarily in **Flows** to call Apex logic.

◆ Example 1: Simple Account Name Updater

✓ Use Case:

Update account names by appending "-Updated" from a Flow.

🧠 Apex Code:

```
apex

public class AccountUpdater {
    public class AccountWrapper {
```

```
@InvocableVariable
public Id accountId;

@InvocableVariable
public String suffix;
}

@InvocableMethod(label='Update Account Names' description='Appends text to account names')
public static void updateAccounts(List<AccountWrapper> inputList) {
    List<Account> accountsToUpdate = new List<Account>();

    for (AccountWrapper input : inputList) {
        Account acc = [SELECT Id, Name FROM Account WHERE Id = :input.accountId LIMIT 1];
        acc.Name += input.suffix;
        accountsToUpdate.add(acc);
    }

    update accountsToUpdate;
}
}
```

✓ In Flow:

- Add an Apex Action element.
- Select **AccountUpdater.updateAccounts**.
- Provide **accountId** and **suffix** as input variables.

◆ Example 2: Return Data to Flow

✓ Use Case:

Search for Contacts by email and return their names.

🧠 Apex Code:

```
apex

public class ContactSearcher {

    public class Input {
        @InvocableVariable
        public String email;
    }

    public class Output {
        @InvocableVariable
        public String contactName;
    }

    @InvocableMethod(label='Find Contacts By Email')
    public static List<Output> findContacts(List<Input> inputList) {
        List<Output> results = new List<Output>();

        for (Input input : inputList) {
            List<Contact> contacts = [SELECT Name FROM Contact WHERE Email = :input.email];
            for (Contact c : contacts) {
                Output o = new Output();
                o.contactName = c.Name;
                results.add(o);
            }
        }

        return results;
    }
}
```

```
}
}
```

✓ In Flow:

- Provide an email address via input.
- Receive and use the returned list of contact names.

◆ Example 3: Creating Custom Object Records

✓ Use Case:

Create **Case** records from Flow inputs.

🧠 Apex Code:

```
apex

public class CaseCreator {

    public class CaseInput {
        @InvocableVariable
        public String subject;

        @InvocableVariable
        public String description;
    }

    @InvocableMethod(label='Create Cases')
    public static void createCases(List<CaseInput> inputs) {
        List<Case> cases = new List<Case>();
        for (CaseInput input : inputs) {
            cases.add(new Case(Subject = input.subject, Description = input.description));
        }

        insert cases;
    }
}
```

◆ Example 4: Sending Custom Emails

✓ Use Case:

Send email using Flow inputs like recipient, subject, and body.

🧠 Apex Code:

```
apex

public class EmailSender {

    public class EmailInput {
        @InvocableVariable
        public String toAddress;

        @InvocableVariable
```

```

    public String subject;

    @InvocableVariable
    public String body;
}

@InvocableMethod(label='Send Custom Email')
public static void sendEmails(List<EmailInput> inputs) {
    List<Messaging.SingleEmailMessage> emails = new List<Messaging.SingleEmailMessage>();

    for (EmailInput input : inputs) {
        Messaging.SingleEmailMessage mail = new Messaging.SingleEmailMessage();
        mail.setToAddresses(new List<String>{input.toAddress});
        mail.setSubject(input.subject);
        mail.setPlainTextBody(input.body);
        emails.add(mail);
    }

    Messaging.sendEmail(emails);
}
}

```

◆ Summary Table

Feature	Details
Annotation	@InvocableMethod
Required Method Properties	public or global , static , accepts a list
Data Classes	Must use @InvocableVariable for fields in custom input/output classes
Use Case	Make Apex usable from Flow / Process Builder
Return Type	void or List<CustomOutputClass>

If you need **more advanced examples** (like handling errors, calling REST API, or chaining Apex in Flow), feel free to ask!



explain IVisualforce page
in detail with multiple examples also explain the examples in salesforce apex

In **Salesforce**, a **Visualforce Page** is a **markup language similar to HTML** that allows developers to build **custom user interfaces (UIs)** for desktop and mobile applications. Visualforce integrates tightly with the **Apex backend**, enabling you to access Salesforce data, perform business logic, and create powerful and customized UI elements.

◆ What is a Visualforce Page?

A **Visualforce Page** consists of:

- Markup: Using **<apex: ...> tags** (like HTML).
- Optional controller: Either a **standard controller**, **custom controller**, or **controller extension** written in Apex.

- Data Binding: Connects UI to Salesforce data.
- Executes on the server and generates HTML for the client browser.

◆ Basic Syntax

```
html

<apex:page>
  Hello, Visualforce!
</apex:page>
```

This renders a simple message in the browser.

◆ Components of Visualforce

Component	Description
<code><apex:page></code>	Root tag of a Visualforce page
<code><apex:form></code>	Allows form submission
<code><apex:inputText></code>	Takes input from the user
<code><apex:commandButton></code>	Button to invoke controller actions
<code><apex:pageBlock></code>	Styled container for content
<code><apex:dataTable></code>	Displays data in table format

◆ Types of Controllers

Type	Description
Standard Controller	Uses built-in CRUD functionality for standard or custom objects
Custom Controller	Written in Apex, full control of behavior
Controller Extension	Extends standard controller, adds custom logic

✓ Examples

◆ Example 1: Using Standard Controller (No Apex Needed)

◆ Visualforce Page: Display Account Details

```
html

<apex:page standardController="Account">
  <h1>Account Details</h1>
  <p>Account Name: {!Account.Name}</p>
  <p>Account Industry: {!Account.Industry}</p>
</apex:page>
```

◆ Explanation:

- Uses the standard controller for the **Account** object.
- Uses merge syntax `{!Account.Name}` to access fields.

🔗 **Usage:** Open this page with a record ID, like:

```
bash

/apex/AccountPage?id=001XXXXXXXXXXXX
```

◆ Example 2: Using Custom Controller to Display and Update Contact

◆ Apex Controller

```
apex

public class ContactController {
    public Contact contact { get; set; }

    public ContactController() {
        contact = new Contact();
    }

    public PageReference saveContact() {
        insert contact;
        return null;
    }
}
```

◆ Visualforce Page

```
html

<apex:page controller="ContactController">
    <apex:form>
        <apex:pageBlock title="New Contact">
            <apex:pageBlockSection>
                <apex:inputField value="{!contact.FirstName}" />
                <apex:inputField value="{!contact.LastName}" />
                <apex:inputField value="{!contact.Email}" />
            </apex:pageBlockSection>
            <apex:commandButton value="Save" action="{!saveContact}" />
        </apex:pageBlock>
    </apex:form>
</apex:page>
```

◆ Explanation:

- A custom Apex controller initializes a Contact.
- On form submission, the contact is inserted.
- Fields are bound to the `contact` property.

◆ Example 3: Display List of Accounts using Apex Controller

◆ Apex Controller

```
apex

public class AccountListController {
    public List<Account> accounts { get; set; }

    public AccountListController() {
        accounts = [SELECT Name, Industry FROM Account LIMIT 10];
    }
}
```

◆ Visualforce Page

```
html

<apex:page controller="AccountListController">
    <h2>Accounts</h2>
    <apex:pageBlock title="Account List">
        <apex:pageBlockTable value="{!accounts}" var="acc">
            <apex:column value="{!acc.Name}" />
            <apex:column value="{!acc.Industry}" />
        </apex:pageBlockTable>
    </apex:pageBlock>
</apex:page>
```

◆ Explanation:

- The controller queries accounts.
- The page displays them in a styled table using `<apex:pageBlockTable>`.

◆ Example 4: Controller Extension (Add Logic to Standard Controller)

◆ Apex Extension

```
apex

public class AccountExtension {
    public Account acc { get; set; }

    public AccountExtension(ApexPages.StandardController std) {
        this.acc = (Account) std.getRecord();
    }

    public void changeName() {
        acc.Name += ' - Modified';
    }
}
```

◆ Visualforce Page

```
html

<apex:page standardController="Account" extensions="AccountExtension">
    <apex:form>
        <apex:pageBlock title="Edit Account">
            <apex:inputField value="{!Account.Name}" />
        </apex:pageBlock>
    </apex:form>
</apex:page>
```

```
<apex:commandButton value="Modify Name" action="{!changeName}" rerender="pb" />
</apex:pageBlock>
</apex:form>
</apex:page>
```

◆ Visualforce + Apex Integration Summary

Visualforce Feature	Connected Apex Feature
{!expression}	Binds to Apex property
<apex:commandButton>	Calls Apex method
controller="..."	Custom logic in Apex
standardController	Auto CRUD support
controller extension	Enhance standard logic

✓ When to Use Visualforce

- Highly customized UI needed (e.g. wizard forms).
- Want full control over HTML/CSS (compared to Lightning).
- Integrating with Apex for custom logic.

← END Conclusion

Visualforce is a powerful, server-side templating system integrated with Apex that lets you:

- Display and manipulate Salesforce data.
- Design interactive UIs with custom logic.
- Extend standard Salesforce behavior.

Would you like advanced examples such as pagination, file upload, or Visualforce email templates?



yes

Great! Let's dive into some **advanced Visualforce examples** using **Apex**, covering:

1. ✓ Pagination
2. ✓ File Upload to Attachments
3. ✓ Visualforce Email Template
4. ✓ Visualforce + Chart
5. ✓ Visualforce PDF Generation

◆ 1. Pagination Example (Custom Page Navigation)

◆ Apex Controller

```
apex

public class AccountPaginationController {
    public Integer pageSize = 5;
    public Integer pageNumber = 0;
    public List<Account> accounts { get; set; }

    public Integer totalRecords {
        get {
            return [SELECT COUNT() FROM Account];
        }
    }

    public void fetchAccounts() {
        accounts = [SELECT Name, Industry FROM Account LIMIT :pageSize OFFSET :pageNumber *
        pageSize];
    }

    public void next() {
        pageNumber++;
        fetchAccounts();
    }

    public void previous() {
        if (pageNumber > 0) pageNumber--;
        fetchAccounts();
    }

    public AccountPaginationController() {
        fetchAccounts();
    }
}
```

◆ Visualforce Page

```
html

<apex:page controller="AccountPaginationController">
    <apex:form>
        <apex:pageBlock title="Paginated Accounts">
            <apex:pageBlockTable value="{!accounts}" var="acc">
                <apex:column value="{!acc.Name}" />
                <apex:column value="{!acc.Industry}" />
            </apex:pageBlockTable>

            <apex:commandButton value="Previous" action="{!previous}" rerender="block" />
            <apex:commandButton value="Next" action="{!next}" rerender="block" />
        </apex:pageBlock>
    </apex:form>
</apex:page>
```

◆ 2. File Upload to Attachments (Classic)

◆ Apex Controller

```
apex

public class FileUploadController {
    public Attachment myAttachment { get; set; }

    public FileUploadController() {
        myAttachment = new Attachment();
    }

    public PageReference save() {
        myAttachment.ParentId = '001XXXXXXXXXXXX'; // Replace with actual record ID
        insert myAttachment;
        return null;
    }
}
```

◆ Visualforce Page

```
html

<apex:page controller="FileUploadController">
    <apex:form enctype="multipart/form-data">
        <apex:pageBlock title="Upload File">
            <apex:inputFile value="{!myAttachment.Body}" filename="{!myAttachment.Name}" />
            <apex:commandButton value="Upload" action="{!save}" />
        </apex:pageBlock>
    </apex:form>
</apex:page>
```

⚠ In Lightning Experience, use **ContentVersion** instead of **Attachment**.

◆ 3. Visualforce Email Template

◆ Visualforce Email Template

```
html

<messaging:emailTemplate subject="New Case Created: {!relatedTo.CaseNumber}"
                        recipientType="Contact"
                        relatedToType="Case">

    <messaging:plainTextEmailBody>
Hello {!recipient.Name},

A new case has been created with the following details:

Case Number: {!relatedTo.CaseNumber}
Subject: {!relatedTo.Subject}
Status: {!relatedTo.Status}

Thank you,
Support Team
    </messaging:plainTextEmailBody>
</messaging:emailTemplate>
```

This is used for **email alerts**, **workflow**, or **Flow email actions**.

◆ 4. Visualforce Chart Using Google Charts

◆ Apex Controller

```
apex

public class ChartController {
    public List<Account> accounts { get; set; }

    public ChartController() {
        accounts = [SELECT Name, AnnualRevenue FROM Account LIMIT 10];
    }
}
```

◆ Visualforce Page with Google Chart

```
html

<apex:page controller="ChartController">
    <script type="text/javascript" src="https://www.gstatic.com/charts/loader.js"></script>
    <script>
        google.charts.load("current", {packages:["corechart"]});
        google.charts.setOnLoadCallback(drawChart);

        function drawChart() {
            var data = google.visualization.arrayToDataTable([
                ['Account', 'Annual Revenue'],
                <apex:repeat value="{!accounts}" var="acc">
                    ['{!acc.Name}', {!acc.AnualRevenue}],
                </apex:repeat>
            ]);

            var options = {
                title: 'Top 10 Accounts by Revenue',
                pieHole: 0.4,
            };

            var chart = new
            google.visualization.PieChart(document.getElementById('donutchart'));
            chart.draw(data, options);
        }
    </script>

    <div id="donutchart" style="width: 600px; height: 400px;"></div>
</apex:page>
```

◆ 5. Visualforce PDF Generation

◆ Apex Controller

```
apex

public class InvoiceController {
    public Invoice__c invoice { get; set; }

    public InvoiceController() {
        invoice = [SELECT Name, Amount__c, Due_Date__c FROM Invoice__c LIMIT 1];
    }
}
```

◆ Visualforce Page

```
html
```

```
<apex:page controller="InvoiceController" renderAs="pdf">
  <h1>Invoice: {!invoice.Name}</h1>
  <p>Amount: {!invoice.Amount__c}</p>
  <p>Due Date: {!invoice.Due_Date__c}</p>
</apex:page>
```

`renderAs="pdf"` tells Salesforce to render the page as a downloadable PDF file.

✓ Summary of Advanced Use Cases

Use Case	Tools Used
Pagination	OFFSET + Custom Logic
File Upload	<code><apex:inputFile></code> + Attachment/Content
Email Template	<code><messaging:emailTemplate></code>
Charting	Google Charts + Data Repeat
PDF Generation	<code>renderAs="pdf"</code>

Would you like any of these integrated into a complete app (like invoice generator with chart & PDF)? Or want Lightning equivalents as well?



explain Imperative/ promise method
in detail with multiple examples also explain the examples in salesforce apex

In **Salesforce**, the **imperative or promise method** is a way of calling **Apex methods from Lightning Web Components (LWC) manually and programmatically**, giving developers full control over:

- When the method is called
- How the result is handled
- Error handling with `.then()` and `.catch()` (Promises)

◆ What is the Imperative (Promise) Method?

In LWC, Apex can be called in **two ways**:

Type	Description
Wired Method	Automatically invoked by LWC when the component is rendered or data changes
Imperative Method	Manually invoked from JS using Promises (via <code>then()</code> / <code>catch()</code> or <code>async/await</code>)

◆ Syntax of Imperative Apex Method

```
js

import apexMethod from '@salesforce/apex/YourApexClass.methodName';

apexMethod({ param1: value1, param2: value2 })
  .then(result => {
    // handle success
  })
  .catch(error => {
    // handle error
  });
```

⚠ The Apex method must be annotated with `@AuraEnabled(cacheable=false)` and `static`.

✓ Example 1: Simple Apex Call (Get Account List)

◆ Apex Class

```
apex

public with sharing class AccountController {
  @AuraEnabled
  public static List<Account> getAccounts() {
    return [SELECT Id, Name FROM Account LIMIT 5];
  }
}
```

◆ LWC JS (imperative call)

```
js

import { LightningElement, track } from 'lwc';
import getAccounts from '@salesforce/apex/AccountController.getAccounts';

export default class ImperativeExample extends LightningElement {
  @track accounts = [];
  @track error;

  handleClick() {
    getAccounts()
      .then(result => {
        this.accounts = result;
      })
      .catch(error => {
        this.error = error;
        console.error('Error retrieving accounts:', error);
      });
  }
}
```

◆ LWC HTML

```
html

<template>
  <lightning-button label="Get Accounts" onclick={handleClick}></lightning-button>
  <template if:true={accounts}>
    <ul>
      <template for:each={accounts} for:item="acc">
        <li key={acc.Id}>{acc.Name}</li>
      </template>
    </ul>
  </template>
```

```

        </template>
    </ul>
</template>
<template if:true={error}>
    <p>Error: {error.body.message}</p>
</template>
</template>

```

✓ Example 2: Imperative Method With Parameters

◆ Apex

```

apex

public with sharing class ContactController {
    @AuraEnabled
    public static List<Contact> getContactsByAccountId(Id accountId) {
        return [SELECT Id, Name, Email FROM Contact WHERE AccountId = :accountId];
    }
}

```

◆ LWC JS

```

js

import { LightningElement, track } from 'lwc';
import getContactsByAccountId from '@salesforce/apex/ContactController.getContactsByAccountId';

export default class ImperativeWithParams extends LightningElement {
    @track contacts = [];
    @track error;

    accountId = '001XXXXXXXXXXXXXXX'; // Replace with real ID

    handleFetchContacts() {
        getContactsByAccountId({ accountId: this.accountId })
            .then(result => {
                this.contacts = result;
            })
            .catch(error => {
                this.error = error;
            });
    }
}

```

◆ LWC HTML

```

html

<template>
    <lightning-button label="Fetch Contacts" onclick={handleFetchContacts}></lightning-button>
    <template if:true={contacts}>
        <template for:each={contacts} for:item="con">
            <p key={con.Id}>{con.Name} - {con.Email}</p>
        </template>
    </template>
</template>

```

✓ Example 3: Use `async/await` for Cleaner Syntax

```
js
async handleClick() {
  try {
    const result = await getAccounts();
    this.accounts = result;
  } catch (error) {
    this.error = error;
    console.error('Error:', error);
  }
}
```

◆ Key Differences Between Wired vs Imperative

Feature	Wired Method	Imperative Method
Auto-called?	Yes	No, must be called explicitly
Supports parameters?	Yes, but declaratively	Yes, via JS function arguments
Use Case	Auto-fetch, reactive UI	Conditional fetching, button click
Control	Limited	Full control (ideal for dynamic logic)

◆ Summary

- Imperative Apex calls are **manual, programmatic** calls using Promises.
- Excellent for button-triggered logic, conditional data fetch, and advanced flows.
- Uses `.then()`, `.catch()` or `async/await` for clean handling of asynchronous behavior.

Would you like examples of:

- Saving data (create record) with imperative method?
- Using `@AuraEnabled` with complex objects (e.g., custom types)?
- Comparing Apex imperative calls with REST calls from LWC?

Let me know and I can share those too!



yes

Excellent! Let's now go **deeper** into **imperative Apex calls** from Lightning Web Components (LWC) with the following **advanced use cases**:

✓ Advanced Imperative Apex Examples

Covered in this guide:

1. ♦ Create a Record from LWC using Apex
2. ♦ Imperative Apex with Custom Object / Wrapper Class
3. ♦ Compare Imperative Apex Call vs REST API Call
4. ♦ Handle Complex Errors in Imperative Calls
5. ♦ Use Imperative Calls inside `async/await` with Loading Spinners

♦ 1. Create a Record (Imperative Save Operation)

♦ Apex: Insert Contact

```
apex

public with sharing class ContactCreator {
    @AuraEnabled
    public static Contact createContact(Contact con) {
        insert con;
        return con;
    }
}
```

♦ LWC JS

```
js

import { LightningElement, track } from 'lwc';
import createContact from '@salesforce/apex/ContactCreator.createContact';

export default class CreateContactImperative extends LightningElement {
    @track firstName = '';
    @track lastName = '';
    @track email = '';
    @track resultMessage = '';

    handleInputChange(event) {
        const field = event.target.name;
        this[field] = event.target.value;
    }

    handleSave() {
        const newContact = {
            FirstName: this.firstName,
            LastName: this.lastName,
            Email: this.email
        };

        createContact({ con: newContact })
            .then(result => {
                this.resultMessage = 'Contact Created: ' + result.Id;
            })
            .catch(error => {
                this.resultMessage = 'Error: ' + error.body.message;
            });
    }
}
```

♦ LWC HTML

```
html

<template>
  <lightning-input name="firstName" label="First Name" onchange={handleInputChange}>
</lightning-input>
  <lightning-input name="lastName" label="Last Name" onchange={handleInputChange}></lightning-input>
  <lightning-input name="email" label="Email" onchange={handleInputChange}></lightning-input>
  <lightning-button label="Create Contact" onclick={handleSave}></lightning-button>
  <p>{resultMessage}</p>
</template>
```

◆ 2. Imperative Call with Custom Wrapper (Object)

◆ Apex: Wrapper Class

```
apex

public with sharing class ContactService {
    public class ContactWrapper {
        @AuraEnabled public String firstName;
        @AuraEnabled public String lastName;
        @AuraEnabled public String email;
    }

    @AuraEnabled
    public static String saveContact(ContactWrapper cw) {
        Contact c = new Contact(
            FirstName = cw.firstName,
            LastName = cw.lastName,
            Email = cw.email
        );
        insert c;
        return c.Id;
    }
}
```

◆ LWC JS

```
js

import saveContact from '@salesforce/apex/ContactService.saveContact';

const wrapper = {
    firstName: 'John',
    lastName: 'Doe',
    email: 'john@example.com'
};

saveContact({ cw: wrapper })
    .then(result => console.log('Saved contact ID:', result))
    .catch(error => console.error('Error:', error));
```

◆ 3. Imperative Apex Call vs REST API Call

Feature	Imperative Apex	REST API Call (via fetch)
Usage	Call Apex directly	Call external/internal REST service
Authentication	Handled automatically in Apex	Requires Named Credential or session

Feature	Imperative Apex	REST API Call (via fetch)
Syntax	<code>import method from '@salesforce/apex'</code>	<code>fetch('/services/apexrest/...')</code>

◆ REST API Example (call Apex REST class)

```
js

fetch('/services/apexrest/MyCustomREST/', {
  method: 'GET',
  headers: {
    'Content-Type': 'application/json'
  }
})
.then(response => response.json())
.then(data => console.log(data))
.catch(error => console.error(error));
```

◆ 4. Handle Complex Errors (like Validation Errors)

◆ Apex

```
apex

@AuraEnabled
public static void errorExample() {
  // Simulate an error
  if (true) {
    throw new AuraHandledException('Custom validation failed');
  }
}
```

◆ LWC

```
js

errorExample()
  .catch(error => {
    if (error.body && error.body.message) {
      this.errorMessage = error.body.message;
    } else {
      this.errorMessage = 'Unknown error';
    }
  });
```

◆ 5. async/await with Loading Spinner

◆ HTML

```
html

<template>
  <lightning-button label="Load Accounts" onclick={loadAccounts}></lightning-button>
  <template if:true={loading}>
    <lightning-spinner alternative-text="Loading..."></lightning-spinner>
  </template>
</template>
```

```
</template>
</template>
```

◆ JS

```
js

import getAccounts from '@salesforce/apex/AccountController.getAccounts';

export default class SpinnerExample extends LightningElement {
  loading = false;

  async loadAccounts() {
    this.loading = true;
    try {
      const result = await getAccounts();
      console.log('Accounts:', result);
    } catch (error) {
      console.error('Error:', error);
    } finally {
      this.loading = false;
    }
  }
}
```

← END Summary

Use Case	Method
Create Record via Apex	Imperative call with Apex insert
Pass Custom Wrapper Object	Define Apex wrapper + @AuraEnabled
Compare with REST API	REST calls offer more flexibility
Handle Complex Errors	Use <code>.catch()</code> + <code>AuraHandledException</code>
Show Spinner During Call	Use <code>loading</code> flag + <code><lightning-spinner></code>

Would you like:

- A reusable **generic LWC-Apex framework**?
- Calling **external APIs** from Apex and using in LWC?
- Working with **complex data types** like maps/lists?

Let me know — happy to help you go further!



Calling external APIs from Apex and using in LWC

Great! Let's walk through **how to call external APIs from Apex** and then **use the result in LWC via an imperative method**.

✓ Workflow Overview

1.  Apex calls an **external REST API** using `Http` and `HttpRequest`.
2.  Response is parsed in Apex and returned.
3.  LWC calls the Apex method **imperatively** and displays the response.

🔧 Step-by-Step Implementation

♦ 1. Set Up a Named Credential (Required for External APIs)

To securely call external APIs:

1. Go to **Setup** → **Named Credentials**
2. Click "**New Named Credential**"
3. Example config:

- **Label:** OpenWeather
- **Name:** OpenWeather
- **URL:** `https://api.openweathermap.org`
- **Authentication:** Choose **No Authentication** (or as needed)

 Named Credentials are required by Salesforce to whitelist external domains.

♦ 2. Apex: Call External API Using Named Credential

apex

```
public with sharing class WeatherService {
    @AuraEnabled
    public static String getWeather(String city) {
        Http http = new Http();
        HttpRequest req = new HttpRequest();

        // Replace with your API key
        String apiKey = 'your_api_key_here';
        String endpoint = '/data/2.5/weather?q=' + EncodingUtil.urlEncode(city, 'UTF-8') +
            '&appid=' + apiKey + '&units=metric';

        req.setEndpoint('callout:OpenWeather' + endpoint); // Named Credential alias
        req.setMethod('GET');

        HttpResponse res = http.send(req);

        if (res.getStatusCode() == 200) {
            return res.getBody();
        } else {
            throw new AuraHandledException('Error from Weather API: ' + res.getStatusCode());
        }
    }
}
```

◆ 3. LWC: JS (Imperative Call)

```
js

import { LightningElement, track } from 'lwc';
import getWeather from '@salesforce/apex/WeatherService.getWeather';

export default class WeatherApp extends LightningElement {
  @track city = 'Bangalore';
  @track weatherData;
  @track error;

  handleCityChange(event) {
    this.city = event.target.value;
  }

  handleGetWeather() {
    getWeather({ city: this.city })
      .then(result => {
        this.weatherData = JSON.parse(result); // Parse JSON string
        this.error = undefined;
      })
      .catch(error => {
        this.error = error.body.message;
        this.weatherData = undefined;
      });
  }
}
```

◆ 4. LWC: HTML

```
html

<template>
  <lightning-input label="Enter City" onchange={handleCityChange}></lightning-input>
  <lightning-button label="Get Weather" onclick={handleGetWeather}></lightning-button>

  <template if:true={weatherData}>
    <p><strong>City:</strong> {weatherData.name}</p>
    <p><strong>Temperature:</strong> {weatherData.main.temp}°C</p>
    <p><strong>Weather:</strong> {weatherData.weather[0].description}</p>
  </template>

  <template if:true={error}>
    <p style="color:red;">Error: {error}</p>
  </template>
</template>
```

Security Notes

- Use **Named Credential** to **avoid hardcoding** API endpoints and secure external calls.
- If the API requires auth (OAuth2, API keys, etc.), configure that inside the Named Credential.
- Salesforce enforces **callout limits** (100 callouts per transaction).

Alternate Use Case Examples

Use Case	API Source
Get Weather Info	OpenWeather API
Get Stock Price	Yahoo Finance or AlphaVantage
Translate Text	Google Translate API
Send SMS	Twilio
Fetch News	NewsAPI.org

Summary

Step	Tool/Concept
External Call	Apex HttpRequest
Whitelist Endpoint	Named Credential
Send & Receive JSON	Parse in Apex / LWC
Use in UI	LWC Imperative Apex Call

Want More?

Would you like:

- A reusable Apex class for **any REST call** (headers, params, methods)?
- An example to **post data (e.g., form submission) to an external API**?
- A version that works in **Lightning Experience Quick Action**?

Let me know!



explain Wire method as parameter and as method in detail with multiple examples also explain the examples in salesforce apex

In **Salesforce Lightning Web Components (LWC)**, the `@wire` decorator is used to **read Salesforce data reactively** via Apex, or LDS (Lightning Data Service). You can use it in two main ways:

Two Ways to Use `@wire`

Usage Type	Description
As a Property (Parameter)	Automatically receives data and errors in a reactive variable
As a Method	Receives data and error as parameters (more control)

Both use `@wire`, but they differ in how the result is handled.

Common Setup (Apex Class)

Let's assume the following Apex class is used in both examples:

◆ Apex Class

```
apex

public with sharing class AccountService {
    @AuraEnabled(cacheable=true)
    public static List<Account> getAccounts() {
        return [SELECT Id, Name, Industry FROM Account LIMIT 10];
    }
}
```

◆ 1. @wire as a Property (Parameter Binding)

◆ Description:

- The wired method **auto-runs** and **auto-updates** when parameters change.
- Simple to use, **best for read-only or reactive UIs**.

◆ JS (LWC)

```
js

import { LightningElement, wire, track } from 'lwc';
import getAccounts from '@salesforce/apex/AccountService.getAccounts';

export default class WireAsProperty extends LightningElement {
    @wire(getAccounts)
    accounts;
}
```

◆ HTML

```
html

<template>
    <template if:true={accounts.data}>
        <template for:each={accounts.data} for:item="acc">
            <p key={acc.Id}>{acc.Name} - {acc.Industry}</p>
        </template>
    </template>
    <template if:true={accounts.error}>
        <p style="color:red;">Error: {accounts.error.message}</p>
    </template>
</template>
```

Breakdown:

- `@wire(getAccounts)` binds the Apex method response to **accounts**.
- **accounts.data** contains the result.
- **accounts.error** contains the error.

- No need to explicitly call the function.

◆ 2. @wire as a Method (With Callback Handling)

◆ Description:

- Gives you **full control** over **data** and **error**.
- You can preprocess, log, or manipulate results.
- Ideal when you need **custom logic** after data is fetched.

◆ JS (LWC)

```
js

import { LightningElement, wire, track } from 'lwc';
import getAccounts from '@salesforce/apex/AccountService.getAccounts';

export default class WireAsFunction extends LightningElement {
  @track accounts = [];
  @track error;

  @wire(getAccounts)
  wiredAccounts({ data, error }) {
    if (data) {
      this.accounts = data.map(acc => ({
        ...acc,
        displayName: acc.Name + ' (' + acc.Industry + ')'
      }));
    } else if (error) {
      this.error = error;
    }
  }
}
```

◆ HTML

```
html

<template>
  <template if:true={accounts}>
    <ul>
      <template for:each={accounts} for:item="acc">
        <li key={acc.Id}>{acc.displayName}</li>
      </template>
    </ul>
  </template>
  <template if:true={error}>
    <p>Error: {error.message}</p>
  </template>
</template>
```

Difference Summary

Feature	@wire as Property	@wire as Method
Simplicity	Easier, declarative	More control, logic flexibility

Feature	@wire as Property	@wire as Method
Data Transformation	Not possible directly	Can transform in if (data) block
Error Handling	Must check .error	Full error handling via code
Recommended for	Simple display	Conditional logic, dynamic handling

✓ Bonus Example: @wire with Dynamic Parameters

◆ Apex Method

```
apex

public with sharing class ContactService {
    @AuraEnabled(cacheable=true)
    public static List<Contact> getContactsByAccount(Id accountId) {
        return [SELECT Id, Name, Email FROM Contact WHERE AccountId = :accountId];
    }
}
```

◆ JS (Dynamic Parameter)

```
js

import { LightningElement, wire, track } from 'lwc';
import getContactsByAccount from '@salesforce/apex/ContactService.getContactsByAccount';

export default class WireWithParams extends LightningElement {
    @track accountId = '001XXXXXXXXXXXX'; // Dynamically set

    @wire(getContactsByAccount, { accountId: '$accountId' })
    contacts;
}
```

Use **\$accountId** to make it reactive — changes to **accountId** will rerun the method.

✓ Summary Table

Concept	Explanation
@wire decorator	Connects Apex to LWC declaratively
As Property	Auto-binding of result to variable, simpler
As Method	Callback-style, gives control over data/error
cacheable=true	Required for @wire to support caching & reactivity
Reactive Params	Use \$param syntax to trigger wire refresh on value change

Would you like to:

- Combine **@wire** with **lightning-datatable**?
- Use **@wire** with LDS (**getRecord**, **getObjectInfo**)?
- Explore performance tips for **@wire**?

Let me know what to explain next!



difference between with sharing and without sharing in tabular format

Here's a clear **tabular comparison** between **with sharing** and **without sharing** keywords in **Salesforce Apex classes**:

Difference Between with sharing VS without sharing

Feature	with sharing	without sharing
Definition	Enforces the current user's sharing rules	Ignores the current user's sharing rules
Purpose	Ensures Apex respects the organization's data security	Used when full access to records is required
Data Visibility	User sees only records they have access to	User sees all records , regardless of sharing
Use Case	Most Apex classes (e.g., controllers)	System-level operations, admin jobs, batch processes
Security Compliance	Aligns with organization-wide defaults (OWD) , roles, and sharing rules	Bypasses sharing settings – use with caution
Inheritance Behavior	Inherited by inner classes unless overridden	Inherited by inner classes unless overridden
Default (if not specified)	Defaults to without sharing	Same (no keyword = without sharing)
Can be Mixed in Code?	Yes, outer class can use with sharing , inner without	Yes, but design should be clear to avoid confusion
Used in	Triggers, Controllers, Visualforce/Aura/LWC Apex classes	Scheduled classes, Queueable, Batch, Utility classes

Example

◆ With Sharing (User-level access)

```
apex

public with sharing class AccountController {
    public List<Account> getAccounts() {
        return [SELECT Id, Name FROM Account]; // Returns only accounts visible to user
    }
}
```

◆ Without Sharing (System-level access)

apex

```
public without sharing class AdminController {
    public List<Account> getAllAccounts() {
        return [SELECT Id, Name FROM Account]; // Returns all accounts
    }
}
```

⚠ Important Notes

- **with sharing is recommended** for Apex controllers used in UI (LWC, Aura, VF) to respect record-level access.
- **without sharing can cause security risks** if misused — always validate record ownership in such cases.
- Apex `@AuraEnabled` or `@RemoteAction` methods **respect sharing only if class is marked with sharing**.

Let me know if you'd like a real-life scenario comparison or want to see how it works in LWC!



difference between `@AuraEnabled` and `@AuraEnabled(cacheable=true)` in tabular format

Here's a clear **tabular comparison** between `@AuraEnabled` and `@AuraEnabled(cacheable=true)` in **Salesforce Apex**:

🔍 Difference Between `@AuraEnabled` VS `@AuraEnabled(cacheable=true)`

Feature	<code>@AuraEnabled</code>	<code>@AuraEnabled(cacheable=true)</code>
Definition	Exposes Apex method to Lightning (Aura/LWC)	Exposes Apex method with client-side caching
Caching	❌ No caching – always makes a server call	✅ Caches the result – faster performance
Server Round-trip	Always performed	Only performed if cache is not available
Return Type	Can be void or return any serializable type	Must return a serializable value (e.g., List , Map , object, etc.)
Side Effects (e.g., DML)	✅ Can perform DML, callouts, etc.	❌ Cannot perform DML, callouts, or changes
Reactivity in LWC	Not reactive – use with imperative methods	Reactive when used with <code>@wire</code> in LWC
Use in <code>@wire</code>	❌ Not allowed	✅ Required for <code>@wire</code> usage
Performance	Slower (no cache)	Faster due to local cache
Typical Use Case	Actions like saving data, callouts, processing	Read-only data retrieval (e.g., fetching records)

✅ Example Without Caching (Imperative Call Only)

```
apex

@AuraEnabled
public static void saveContact(Contact con) {
    insert con; // DML is allowed
}
```

Used like:

```
js

saveContact({ con: this.contactData })
    .then(result => { /* handle result */ })
```

✓ Example With Caching (Used in @wire)

```
apex

@AuraEnabled(cacheable=true)
public static List<Account> getAccounts() {
    return [SELECT Id, Name FROM Account LIMIT 10]; // Read-only
}
```

Used like:

```
js

@wire(getAccounts) accounts;
```

⚠ Important Notes

Constraint	@AuraEnabled(cacheable=true) must not perform:
✓ DML	✗ INSERT, UPDATE, DELETE
✓ Callouts	✗ HTTP requests, external APIs
✓ Side Effects	✗ Any action that changes server-side data

🧠 Summary

Use @AuraEnabled when:

- You need to **modify data** (DML, callouts, etc.)
- You're using **imperative Apex** in LWC or Aura

Use @AuraEnabled(cacheable=true) when:

- You need **read-only data**
- You want **reactivity** with @wire
- You want **better performance** via caching

Let me know if you want real LWC code using both types!



yes

Great! Below are **two full LWC examples**:

1. Using `@AuraEnabled(cacheable=true)` with `@wire` (read-only & reactive)
2. Using `@AuraEnabled` with **imperative call** (for DML)

✓ 1. Example: `@AuraEnabled(cacheable=true)` with `@wire`

Use case: Fetch and display a list of Accounts (read-only)

◆ Apex Class

```
apex

public with sharing class AccountService {
    @AuraEnabled(cacheable=true)
    public static List<Account> getAccounts() {
        return [SELECT Id, Name FROM Account LIMIT 5];
    }
}
```

◆ LWC JS: `accountList.js`

```
js

import { LightningElement, wire } from 'lwc';
import getAccounts from '@salesforce/apex/AccountService.getAccounts';

export default class AccountList extends LightningElement {
    @wire(getAccounts) accounts; // Reactive & auto-updates if data changes
}
```

◆ LWC HTML: `accountList.html`

```
html

<template>
    <template if:true={accounts.data}>
        <lightning-card title="Accounts">
            <ul>
                <template for:each={accounts.data} for:item="account">
                    <li key={account.Id}>{account.Name}</li>
                </template>
            </ul>
        </lightning-card>
    </template>
    <template if:true={accounts.error}>
```

```
<p style="color:red;">Error: {accounts.error.message}</p>
</template>
</template>
```

✓ 2. Example: @AuraEnabled with Imperative Call

Use case: Insert a new Contact record using a button

◆ Apex Class

```
apex

public with sharing class ContactService {
    @AuraEnabled
    public static void saveContact(Contact contactObj) {
        insert contactObj;
    }
}
```

◆ LWC JS: createContact.js

```
js

import { LightningElement, track } from 'lwc';
import saveContact from '@salesforce/apex/ContactService.saveContact';

export default class CreateContact extends LightningElement {
    @track contact = { FirstName: '', LastName: '', Email: '' };
    @track message = '';

    handleChange(event) {
        const field = event.target.name;
        this.contact[field] = event.target.value;
    }

    handleSave() {
        saveContact({ contactObj: this.contact })
            .then(() => {
                this.message = 'Contact saved successfully!';
            })
            .catch(error => {
                this.message = 'Error: ' + error.body.message;
            });
    }
}
```

◆ LWC HTML: createContact.html

```
html

<template>
    <lightning-card title="Create Contact">
        <lightning-input name="FirstName" label="First Name" onchange={handleChange}>
    </lightning-input>
        <lightning-input name="LastName" label="Last Name" onchange={handleChange}></lightning-
input>
        <lightning-input name="Email" label="Email" type="email" onchange={handleChange}>
    </lightning-input>
</template>
```

```

<lightning-button label="Save Contact" onclick={handleSave} class="slds-m-top_medium">
</lightning-button>

<p class="slds-m-top_small">{message}</p>
</lightning-card>
</template>

```

Summary Table

Feature	@AuraEnabled(cacheable=true)	@AuraEnabled
Usage in LWC	@wire() reactive data	Imperative method().then() calls
Performs DML/callout	✗ No	✓ Yes
Use Case	Read-only, fast, reactive components	Form submissions, create/update logic
Syntax Safety	Must return serializable (List, Object)	Can return void, Boolean, etc.

Would you like a combined example that fetches + saves data in the same component?



difference between Imperative/ promise method
and Wire method in tabular format

Here's a clear **tabular comparison** between **Imperative (Promise-based)** method and **Wire** method in **Salesforce Lightning Web Components (LWC)**:

Difference Between Imperative vs Wire Method

Feature / Aspect	Wire Method (@wire)	Imperative Method (Promise-based)
Definition	Reactive way to call Apex or Lightning Data Service	Manual call to Apex using a Promise
Syntax	Uses @wire decorator	Uses standard function call with .then() / async-await
Execution Timing	Auto-invoked on component load or when reactive parameters change	Called only when explicitly invoked by user action
Control Over Execution	✗ No (auto-run)	✓ Full control (e.g., on button click)
Reactivity	✓ Reactive to data/param changes	✗ Not reactive
Caching Support	✓ Yes (requires @AuraEnabled(cacheable=true))	✗ No caching
Supports DML / Callouts	✗ No – must be cacheable and read-only	✓ Yes – can do DML, callouts, etc.
Return Type Required	Must return serializable type (e.g., List, Map, Object)	Can return any type or void
Error Handling	Via error property in wired result	Via .catch() or try...catch

Feature / Aspect	Wire Method (@wire)	Imperative Method (Promise-based)
Common Use Case	Fetching read-only data (e.g., Account list, config settings)	Submitting forms, saving records, conditional logic
Example Use Case	<code>@wire(getAccounts)</code> to list accounts	<code>saveContact(contactObj)</code> on button click
Performance	Faster due to automatic caching	Slower for repeated calls (no caching)

✓ Example Summary

◆ Wire Method

```
js
@wire(getAccounts) accounts;
```

Used for **read-only, reactive data fetching** (e.g., displaying account names)

◆ Imperative Method

```
js
handleClick() {
  saveContact({ contactObj: this.contact })
    .then(() => { /* success */ })
    .catch(error => { /* handle error */ });
}
```

Used for **user actions** like creating or updating records

🧠 When to Use Which?

Scenario	Use @wire	Use Imperative
Auto-fetching data on load	✓ Yes	✗ Not preferred
Fetching only after user action	✗ No	✓ Yes
Performing DML (insert/update)	✗ Not allowed	✓ Yes
Callout to external service	✗ Not allowed	✓ Yes
Reactive to parameter change	✓ Yes	✗ No

Let me know if you'd like examples of both in the **same component** for better comparison!



yes

Great! Let's create a **single Lightning Web Component (LWC)** that demonstrates **both @wire and imperative method** usage — side by side.

Use Case

- Use `@wire` to **fetch a list of Accounts** (read-only, reactive).
- Use **imperative method** to **insert a new Contact** via button click.

◆ Apex Class: AccountContactService.cls

```
apex

public with sharing class AccountContactService {

    // Read-only, cacheable method for wire
    @AuraEnabled(cacheable=true)
    public static List<Account> getAccounts() {
        return [SELECT Id, Name FROM Account LIMIT 5];
    }

    // Imperative method for DML (insert)
    @AuraEnabled
    public static void createContact(Contact contactObj) {
        insert contactObj;
    }
}
```

◆ LWC JS: accountAndContact.js

```
js

import { LightningElement, wire, track } from 'lwc';
import getAccounts from '@salesforce/apex/AccountContactService.getAccounts';
import createContact from '@salesforce/apex/AccountContactService.createContact';

export default class AccountAndContact extends LightningElement {
    // Wire for reactive fetching of accounts
    @wire(getAccounts) accounts;

    @track contact = { FirstName: '', LastName: '', Email: '' };
    @track message = '';

    // Handle input change for contact
    handleInputChange(event) {
        const field = event.target.name;
        this.contact[field] = event.target.value;
    }

    // Call Apex imperatively
    handleSaveContact() {
        createContact({ contactObj: this.contact })
            .then(() => {
                this.message = '✅ Contact saved successfully!';
            })
            .catch(error => {
                this.message = '❌ Error: ' + error.body.message;
            });
    }
}
```

◆ LWC HTML: accountAndContact.html

```
html

<template>
  <lightning-card title="Accounts (Wire Method)">
    <template if:true={accounts.data}>
      <ul class="slds-m-left_medium">
        <template for:each={accounts.data} for:item="acc">
          <li key={acc.Id}>{acc.Name}</li>
        </template>
      </ul>
    </template>
    <template if:true={accounts.error}>
      <p style="color:red;">Error: {accounts.error.message}</p>
    </template>
  </lightning-card>

  <lightning-card title="Create Contact (Imperative)">
    <lightning-input name="FirstName" label="First Name" onchange={handleInputChange}>
  </lightning-input>
    <lightning-input name="LastName" label="Last Name" onchange={handleInputChange}>
  </lightning-input>
    <lightning-input name="Email" type="email" label="Email" onchange={handleInputChange}>
  </lightning-input>

    <lightning-button class="slds-m-top_medium" label="Save Contact" onclick=
{handleSaveContact}></lightning-button>
    <p class="slds-m-top_small">{message}</p>
  </lightning-card>
</template>
```

✓ Summary of What's Demonstrated

Method	Purpose	How it's used
@wire(getAccounts)	Read-only, auto data fetch	Displays list of Accounts reactively
createContact()	Imperative, on button click	Saves new contact using Apex DML

Let me know if you want a version with refresh using `refreshApex()` or if you'd like this component to be shown in a modal!



Difference between flows like record triggered flow, scheduled flow, screen flow, auto launched flow in tabular format

Here is a detailed **tabular comparison** of different types of **Salesforce Flows**:

🔍 Difference Between Record-Triggered, Scheduled, Screen, and Auto-Launched Flows

Feature / Aspect	Record-Triggered Flow	Scheduled Flow	Screen Flow	Auto-Launched Flow
Triggered by	Changes to a Salesforce record (Create, Update, Delete)	Specific date/time or recurring schedule	User interaction via UI	Programmatic or manual invocation (e.g., Apex, Process Builder)
User Interaction (UI)	✗ No	✗ No	✓ Yes (can display screens to users)	✗ No
Can be Launched by User?	✗ No (automatic on record change)	✗ No	✓ Yes (via button, Lightning page, etc.)	✗ No (only system/integration initiated)
Common Use Case	Update related records, send notifications	Send scheduled emails, auto data cleanup	Wizard-like forms, guided data entry	Invoked via Apex or other flows for background tasks
Supports Screens	✗ No	✗ No	✓ Yes	✗ No
Can Perform DML?	✓ Yes	✓ Yes	✓ Yes	✓ Yes
Supports Loops & Decisions	✓ Yes	✓ Yes	✓ Yes	✓ Yes
Supports Before-Save Triggering	✓ Yes (for updates only)	✗ No	✗ No	✗ No
Invocation Mechanism	Automatically based on record change	Time-based scheduler	Manually from UI (e.g., Button, Tab, App Page)	Invoked from Apex, Process Builder, or other flows
Performance Impact	Medium to High (depends on logic complexity)	Low to Medium	Depends on user interaction and screen usage	Medium

✓ Summary Table

Flow Type	User Input	Triggered By	Runs On Schedule	Manual Launch	Apex Invocation	DML Support
Record-Triggered Flow	✗	Record Create/Update/Delete	✗	✗	✗	✓
Scheduled Flow	✗	Scheduled Time	✓	✗	✗	✓
Screen Flow	✓	User Interaction	✗	✓	✗	✓
Auto-Launched Flow	✗	Programmatic or Flow	✗	✗	✓	✓

🧠 When to Use Each:

Use Case	Best Flow Type
Update a related record when one is edited	Record-Triggered Flow
Send weekly report or cleanup tasks	Scheduled Flow
Guide users through a step-by-step form	Screen Flow
Reusable logic for multiple flows or Apex	Auto-Launched Flow

Let me know if you'd like a flow diagram or visual comparison too!



How to send email all types in flows in apex

In **Salesforce**, you can send emails through **Flows** and **Apex**, and there are multiple ways depending on your requirements. Below is a complete breakdown of **how to send emails using all types of flows** and **Apex**, with explanations and examples:

◆ 1. Sending Email in Flows

✓ Methods:

Flow Type	Can Send Email?	How to Send
Screen Flow	✓ Yes	Use Email Alert or Send Email Action
Record-Triggered Flow	✓ Yes	Use Email Alert or Send Email Action
Scheduled Flow	✓ Yes	Use Email Alert or Send Email Action
Auto-Launched Flow	✓ Yes	Use Email Alert , Send Email , or call Apex

◆ A. Using Email Alert in Flow

📌 Requires a **Workflow Email Alert** created from Setup.

✓ Steps:

1. Go to **Setup** → **Email Alerts** → Create a new alert.
2. In Flow, use **Action** → **Email Alert**
3. Select your email alert and related record.

✓ Works in all flow types.

◆ B. Using "Send Email" Action (Core Action)

📌 Directly send an email by entering subject, body, and recipients.

✓ Steps:

1. In Flow, click on + → **Action** → **Send Email**

2. Set:

- **To Address:** email address or variable
- **Subject:** text or dynamic
- **Body:** plain text or HTML

✓ Works in **Screen**, **Record-Triggered**, **Scheduled**, and **Auto-Launched** flows.

◆ C. Using Apex Action to Send Complex Emails

You can create an **Apex Invocable Method** to send more customized emails (e.g., HTML, attachments).

◆ 2. Sending Email in Apex (All Types)

✓ Using `Messaging.sendEmail()`

There are two primary ways:

◆ A. Simple Text Email (`SingleEmailMessage`)

apex

```
Messaging.SingleEmailMessage mail = new Messaging.SingleEmailMessage();
mail.setToAddresses(new String[] { 'test@example.com' });
mail.setSubject('Hello from Apex');
mail.setPlainTextBody('This is a test email sent from Apex.');
Messaging.sendEmail(new Messaging.SingleEmailMessage[] { mail });
```

◆ B. HTML Email with Templates

apex

```
Messaging.SingleEmailMessage mail = new Messaging.SingleEmailMessage();
mail.setTemplateId('YOUR_TEMPLATE_ID');
mail.setTargetObjectId('CONTACT_OR_USER_ID'); // Needed for merge fields
mail.setSaveAsActivity(false);
Messaging.sendEmail(new Messaging.SingleEmailMessage[] { mail });
```

◆ C. Bulk Emails (Batch/Looped)

apex

```
List<Messaging.SingleEmailMessage> mails = new List<Messaging.SingleEmailMessage>();
for (String email : emailList) {
    Messaging.SingleEmailMessage mail = new Messaging.SingleEmailMessage();
    mail.setToAddresses(new String[] { email });
    mail.setSubject('Bulk Email');
    mail.setPlainTextBody('Hello from Apex');
    mails.add(mail);
}
```

```
}
Messaging.sendEmail(mails);
```

◆ D. Using Apex from Flow

You can expose the email logic from Apex to Flow like this:

Apex Class:

```
apex

public class FlowEmailSender {
    @InvocableMethod(label='Send Custom Email')
    public static void sendEmail(List<EmailRequest> requests) {
        List<Messaging.SingleEmailMessage> mails = new List<Messaging.SingleEmailMessage>();
        for (EmailRequest req : requests) {
            Messaging.SingleEmailMessage mail = new Messaging.SingleEmailMessage();
            mail.setToAddresses(new String[] { req.toEmail });
            mail.setSubject(req.subject);
            mail.setPlainTextBody(req.body);
            mails.add(mail);
        }
        Messaging.sendEmail(mails);
    }
}

public class EmailRequest {
    @InvocableVariable(label='To Email')
    public String toEmail;

    @InvocableVariable(label='Subject')
    public String subject;

    @InvocableVariable(label='Body')
    public String body;
}
}
```

✓ This can be used in **Auto-Launched Flows** via Apex Action.

✓ Summary Table

Method	Used In	Supports Templates	HTML Body	Attachments	When to Use
Email Alert (Flow)	All Flows	✓ Yes	✗ No	✗ No	Standard alerts
Send Email Action	All Flows	✗ No	✓ Yes	✗ No	Quick manual messages
Apex sendEmail()	Apex, Flow via Apex	✓ Yes	✓ Yes	✓ Yes	Complex or dynamic emails
Email Template + Apex	Apex	✓ Yes	✓ Yes	✓ Yes	Personalized content using templates

Let me know if you want:

- An example for sending **email with attachments**.
- How to send emails using **Lightning Email Templates**.

- How to **test email sending in Apex unit tests**.



Attachment and lwc , explain all the steps

To **handle attachments in LWC (Lightning Web Components)**, we typically upload files using the **lightning-file-upload** component and then store them as **ContentVersion**, **ContentDocumentLink**, or traditional **Attachment** objects depending on your org setup (Salesforce Files vs classic Attachments).

✓ Goal:

Upload, preview, download, and manage **attachments in LWC** and store them in Salesforce.

♦ 1. Setup Required Object Permissions

Ensure the following objects have **Create/Read/Write** access:

- **ContentVersion**
- **ContentDocumentLink**
- **ContentDocument**

♦ 2. LWC File Upload Using **lightning-file-upload**

💡 This stores files in the **ContentVersion** object.

attachmentUploader.html

```
html

<template>
  <lightning-card title="Upload Attachment">
    <lightning-file-upload
      label="Upload File"
      name="fileUploader"
      accept={acceptedFormats}
      record-id={recordId}
      onuploadfinished={handleUploadFinished}
      multiple>
    </lightning-file-upload>
  </lightning-card>

  <template if:true={fileNames.length}>
    <lightning-card title="Uploaded Files">
      <ul>
        <template for:each={fileNames} for:item="file">
          <li key={file}>{file}</li>
        </template>
      </ul>
    </lightning-card>
  </template>
```

```
</template>
</template>
```

attachmentUploader.js

```
js

import { LightningElement, api, track } from 'lwc';

export default class AttachmentUploader extends LightningElement {
  @api recordId; // The record to which files will be attached
  acceptedFormats = ['.pdf', '.png', '.jpg', '.jpeg', '.doc', '.docx'];
  @track fileNames = [];

  handleUploadFinished(event) {
    const uploadedFiles = event.detail.files;
    uploadedFiles.forEach(file => {
      this.fileNames.push(file.name);
    });
  }
}
```

attachmentUploader.js-meta.xml

```
xml

<?xml version="1.0" encoding="UTF-8" ?>
<LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">
  <apiVersion>59.0</apiVersion>
  <isExposed>true</isExposed>
  <targets>
    <target>lightning__RecordPage</target>
    <target>lightning__AppPage</target>
  </targets>
</LightningComponentBundle>
```

◆ 3. Where the File Gets Stored

- When using **lightning-file-upload**, Salesforce stores the file as a **ContentVersion**.
- It automatically links the uploaded file to the **recordId** via **ContentDocumentLink**.

◆ 4. Preview/Download Uploaded Files (Optional)

To preview/download uploaded files, you can use the **ContentDocumentId** or file name and generate a download link:

```
js

// Example for download
window.open(`/sfc/servlet.shepherd/document/download/${contentDocumentId}`, "_blank");
```

Or use **lightning-quick-action-panel** to create a preview modal.

◆ 5. Apex: Retrieve Attached Files

🧠 Get All Files Attached to a Record

```
apex

public with sharing class FileController {
    @AuraEnabled(cacheable=true)
    public static List<ContentDocument> getFiles(Id recordId) {
        List<ContentDocumentLink> links = [
            SELECT ContentDocumentId FROM ContentDocumentLink WHERE LinkedEntityId = :recordId
        ];
        Set<Id> docIds = new Set<Id>();
        for (ContentDocumentLink link : links) {
            docIds.add(link.ContentDocumentId);
        }

        return [SELECT Id, Title, FileExtension, LatestPublishedVersionId FROM ContentDocument
            WHERE Id IN :docIds];
    }
}
```

◆ 6. Display Attached Files in LWC (Optional)

You can call the Apex method via **@wire** or **imperative call** and show it in your LWC component as a list.

◆ 7. Delete File via Apex (Optional)

To delete a file, delete the **ContentDocumentLink** and optionally the **ContentDocument**.

```
apex

@AuraEnabled
public static void deleteFile(Id contentDocumentId) {
    List<ContentDocumentLink> links = [
        SELECT Id FROM ContentDocumentLink WHERE ContentDocumentId = :contentDocumentId
    ];
    delete links;
}
```

✓ Summary

Feature	Approach
Upload files	lightning-file-upload in LWC
File storage	Automatically saved as ContentVersion
Link to record	Handled via ContentDocumentLink
View/download file	Use URL /sfc/servlet.shepherd/document/download/ID
Access in Apex	Query ContentDocumentLink + ContentDocument
Delete file	Delete ContentDocumentLink via Apex

Let me know if you need:

- A **version using classic Attachment object** instead of ContentVersion.
- A **complete zip** of code files.
- A **custom file size or type validator** in JS.

Would you like to also display a file preview thumbnail?



When to use which type of send email options

In Salesforce, there are **multiple ways to send emails**, and choosing the right method depends on your **use case, complexity, automation needs, and UI experience**.

Here's a detailed breakdown of **when to use which type of email sending option**:

✓ Comparison of All Email Options in Salesforce

Email Type	When to Use	Key Features	Customizable UI?	Code Required?
Email Alert (via Workflow / Flow / Process Builder)	Simple notifications on record changes (create/update)	Template-based, no code, automation-friendly	✗ No	✗ No
Apex Messaging.sendEmail()	Advanced or conditional email logic (e.g., loops, dynamic addresses, custom subjects, etc.)	Full control via Apex; can use templates or build content dynamically	✗ No	✓ Yes
Flow – Send Email action	For sending emails inside Flows with dynamic field values	Drag-and-drop in Flow Builder, no Apex needed	✗ No	✗ No
Email Template (used across all)	When the email content needs to be standardized (for users, support, sales etc.)	Use with Alerts, Apex, or Flows	✓ Limited (Visualforce/HTML)	✗ No
Lightning Email Composer (LWC UI)	When user needs to send an email manually via UI (e.g., from a Case or Contact record)	Used in Lightning App, customizable from Activity timeline	✓ Yes	✗ No
Einstein Send Time Optimization (Marketing Cloud)	For email campaigns where best delivery time matters	AI-optimized delivery	✗ No	✗ No
LWC with Apex Email Sending	When you need to send email from a button/action in a	User-driven email send with dynamic logic	✓ Yes	✓ Yes

Email Type	When to Use	Key Features	Customizable UI?	Code Required?
	Lightning Web Component	and Apex backend		

◆ Examples of When to Use Each

Scenario	Best Email Method
Send welcome email when a new Contact is created	Email Alert via Flow or Process Builder
Send complex invoice with PDF attachment	Apex with <code>Messaging.EmailFileAttachment</code>
Let user manually send email from UI	Lightning Email Composer
Marketing or newsletter campaign	Marketing Cloud / Pardot Email
Send email on record status change with dynamic recipients	Flow + Send Email Action OR Apex
Scheduled email reminders (e.g., 7 days before due date)	Scheduled Flow + Email Action
User clicks "Send Quote" button in LWC, email is generated and sent	LWC + Apex <code>Messaging.sendEmail()</code>

Decision Guide

- ◆ **Use Flow/Email Alert** when: no code, simple business logic.
- ◆ **Use Apex** when: dynamic data, attachments, loops, conditional logic.
- ◆ **Use LWC + Apex** when: user-driven action from Lightning UI.
- ◆ **Use Marketing Cloud/Pardot** when: bulk or campaign emails.
- ◆ **Use Lightning Email Composer** when: manual sending via Activity timeline.

Example: Send Email from Apex

apex

```
Messaging.SingleEmailMessage mail = new Messaging.SingleEmailMessage();
mail.setToAddresses(new String[] {'user@example.com'});
mail.setSubject('Invoice Ready');
mail.setPlainTextBody('Your invoice is attached.');
```

```
Messaging.sendEmail(new Messaging.SingleEmailMessage[] {mail});
```

Example: Send Email in Flow

- Use the **"Send Email"** standard flow action.
- Use a **Text Template** for body.
- Set **To Address, Subject**, etc., via variables.

Would you like code examples for:

- Sending email with PDF attachment?
- Sending email using Visualforce template?
- Sending email via Apex triggered by Flow?

Let me know your use case and I'll tailor the best approach.



Sending email via Apex triggered by Flow?

Yes! You can **send an email via Apex that is triggered by a Flow** using an **Apex Invocable Method**. This is ideal when Flow needs to send a complex/dynamic email (e.g. with logic, attachments, formatting) which the built-in Flow **Send Email** action can't handle.

✓ Overview: Flow → Invokes Apex → Apex sends Email

📌 Use Case:

- Flow sends dynamic data (like recipient email, subject, body).
- Apex handles the actual `Messaging.sendEmail()` logic.

🔧 Step-by-Step Implementation

♦ 1. Create Apex Class with `@InvocableMethod`

```
apex

public class FlowEmailSender {

    public class EmailInput {
        @InvocableVariable(required=true)
        public String toAddress;

        @InvocableVariable(required=true)
        public String subject;

        @InvocableVariable(required=true)
        public String body;
    }

    @InvocableMethod(label='Send Email From Flow')
    public static void sendEmail(List<EmailInput> inputs) {
        List<Messaging.SingleEmailMessage> emails = new List<Messaging.SingleEmailMessage>();

        for (EmailInput input : inputs) {
            Messaging.SingleEmailMessage email = new Messaging.SingleEmailMessage();
            email.setToAddresses(new String[] { input.toAddress });
            email.setSubject(input.subject);
            email.setPlainTextBody(input.body);
            emails.add(email);
        }

        if (!emails.isEmpty()) {
```

```

    Messaging.sendEmail(emails);
  }
}

```

♦ 2. Create a Flow (e.g. Record-Triggered Flow)

Steps:

1. Go to **Setup > Flows > New Flow**.
2. Choose **Record-Triggered Flow** or **Screen Flow**.
3. Add an **Action** element:
 - Type: **Apex Action**
 - Search for: **Send Email From Flow**
4. Provide values for:
 - **toAddress** (can be hardcoded or from record field like **Contact.Email**)
 - **subject**
 - **body**

♦ 3. Save and Activate the Flow

Once the Flow runs (on create/update), it will call the Apex class, which sends the email.

Example Use Case

Use case: When a **Case** is updated to "Closed", send an email to the Contact.

- Trigger: Case updated.
- Apex sends email:
 - To: **Case.Contact.Email**
 - Subject: "Your case has been resolved"
 - Body: "Hello {Contact Name}, your case is now resolved. Thank you!"

Benefits of Using Apex in Flow

Benefit	Description
Advanced Logic	You can add conditions, loops, or attachments in Apex.
Rich Email Support	Supports HTML, attachments, templates (with more code).
Reusability	Can be used across multiple flows.
Secure	Email logic stays in Apex, no logic clutter in the flow.

Optional Enhancements

- Send **HTML email** using `setHtmlBody()`.
- Add **CC, BCC, ReplyTo**.
- Send from **Organization-Wide Email Address**.
- Add **attachments** using `EmailFileAttachment`.

Would you like:

- A version that includes **PDF attachments**?
- To send email using a **Visualforce email template** from Apex?
- A version that accepts **multiple recipients**?

Let me know and I'll provide the code.



A version that accepts multiple recipients

Great! Here's a version of the Apex class that supports **sending an email to multiple recipients** from **Flow** using an **Invocable Method**.

Use Case

You want to send the **same email** to **multiple recipients** (To, CC, BCC supported) using a **Flow**, and let **Apex handle the email logic**.

Updated Apex Class: Supports Multiple Recipients

```
apex

public class FlowEmailMultiRecipient {

    public class EmailInput {
        @InvocableVariable(required=true)
        public List<String> toAddresses;

        @InvocableVariable
        public List<String> ccAddresses;

        @InvocableVariable
        public List<String> bccAddresses;

        @InvocableVariable(required=true)
        public String subject;

        @InvocableVariable(required=true)
        public String body;
    }

    @InvocableMethod(label='Send Email to Multiple Recipients')
```

```
public static void sendEmail(List<EmailInput> inputs) {
    List<Messaging.SingleEmailMessage> emails = new List<Messaging.SingleEmailMessage>();

    for (EmailInput input : inputs) {
        Messaging.SingleEmailMessage email = new Messaging.SingleEmailMessage();

        if (input.toAddresses != null) {
            email.setToAddresses(input.toAddresses);
        }

        if (input.ccAddresses != null) {
            email.setCcAddresses(input.ccAddresses);
        }

        if (input.bccAddresses != null) {
            email.setBccAddresses(input.bccAddresses);
        }

        email.setSubject(input.subject);
        email.setPlainTextBody(input.body);

        emails.add(email);
    }

    if (!emails.isEmpty()) {
        Messaging.sendEmail(emails);
    }
}
```

Flow Integration Instructions

1. Create a Flow

- Type: **Record-Triggered Flow** or **Screen Flow**

2. Add Apex Action

- Name: **Send Email to Multiple Recipients**

3. Pass Inputs in Flow

- **toAddresses**: Collection of strings (e.g., use formula fields or hardcoded list like ["a@example.com", "b@example.com"])
- **ccAddresses / bccAddresses**: Optional
- **subject**: e.g. "Project Update"
- **body**: e.g. "Please find the project updates attached."

4. Save & Activate Flow

Example Use Case in Flow

Let's say on Account record update, you want to send an email to:

- All related Contacts' emails

- CC a fixed address (`manager@example.com`)

In Flow:

- Create a **Get Records** to fetch Contacts' emails
- Use a **Collection Variable** for `toAddresses`
- Pass that + subject + body to the Apex action

Notes

- This version sends **one email per input**, so if the Flow passes a list of 3 inputs, 3 separate emails will be sent.
- You can add `setHtmlBody()` instead of `setPlainTextBody()` if you want HTML emails.
- You can add error handling/logging if needed for production.

Would you like:

- To send **different emails** to different groups (customized per group)?
- To send **bulk emails** (e.g., one email with many To recipients)?
- Support for **file attachments**?

Let me know, and I'll guide you accordingly.